**Robot Kinematics: (Latex Equations): For TurtleBot4**

**State vector (pose):**

\mathbf{q}(t) = \begin{bmatrix} x(t) \\ y(t) \\ \theta(t) \end{bmatrix}

Robot pose (state)

$$\mathbf{q}(t) = \begin{bmatrix} x(t) \\ y(t) \\ \theta(t) \end{bmatrix}$$

Position in the world frame plus heading angle $\theta$ (yaw)

**Control input vector:**
\mathbf{u}(t) = \begin{bmatrix} \omega_R(t) \\ \omega_L(t) \end{bmatrix}

Control input vector

$$\mathbf{u}(t) = \begin{bmatrix} \omega_R(t) \\ \omega_L(t) \end{bmatrix}$$

Angular velocities of the right and left wheels respectively

**Linear & angular body-frame velocities:**
v(t) = \frac{r}{2}\bigl(\omega_R + \omega_L\bigr), \qquad
\dot{\theta}(t) = \frac{r}{b}\bigl(\omega_R - \omega_L\bigr)

Linear & angular body-frame velocities

$$v(t) = \frac{r}{2}\left(\omega_R + \omega_L\right), \qquad \dot{\theta}(t) = \frac{r}{b}\left(\omega_R - \omega_L\right)$$

$r$ = wheel radius, $b$ = wheelbase (distance between wheel contact points)

**Continuous-time motion model (ODEs):**
\dot{\mathbf{q}}(t) =
\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}
=
\begin{bmatrix}

```
  v(t)\cos\theta(t) \\
  v(t)\sin\theta(t) \\[4pt]
  \dfrac{r}{b}\bigl(\omega_R(t) - \omega_L(t)\bigr)
\end{bmatrix}
```

State derivative — ODEs

$$\dot{\mathbf{q}}(t) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v(t)\cos\theta(t) \\ v(t)\sin\theta(t) \\ \frac{r}{b}\left(\omega_R(t) - \omega_L(t)\right) \end{bmatrix}$$

Unicycle / nonholonomic constraint: robot cannot move sideways

**Discrete-time state update (Euler integration):**

```
\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix}
=
\begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix}
+
\Delta t
\begin{bmatrix}
  \dfrac{r}{2}(\omega_R^k + \omega_L^k)\,\cos\theta_k \\[8pt]
  \dfrac{r}{2}(\omega_R^k + \omega_L^k)\,\sin\theta_k \\[8pt]
  \dfrac{r}{b}(\omega_R^k - \omega_L^k)
\end{bmatrix}
```

Control inputs → state update at time step $k$

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} + \Delta t \begin{bmatrix} \frac{r}{2}(\omega_R^k + \omega_L^k)\,\cos\theta_k \\ \frac{r}{2}(\omega_R^k + \omega_L^k)\,\sin\theta_k \\ \frac{r}{b}(\omega_R^k - \omega_L^k) \end{bmatrix}$$

$\Delta t$ = control loop period (e.g. 10 ms)

**ICC radius:**

```
R = \frac{b}{2}\cdot\frac{\omega_R + \omega_L}{\omega_R - \omega_L}
\qquad (\omega_R \neq \omega_L)
```

Instantaneous centre of curvature radius

$$R = \frac{b}{2} \cdot \frac{\omega_R + \omega_L}{\omega_R - \omega_L} \qquad (\omega_R \neq \omega_L)$$

**Exact arc update (ICC method):**

```
\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix}
=
\begin{bmatrix}
  x_k + R\bigl(\sin(\theta_k + \Delta\theta) - \sin\theta_k\bigr) \\
  y_k - R\bigl(\cos(\theta_k + \Delta\theta) - \cos\theta_k\bigr) \\
  \theta_k + \Delta\theta
\end{bmatrix},
\quad \Delta\theta = \dot{\theta}\,\Delta t
```

Closed-form arc update (no linearisation error)

$$\begin{bmatrix} x_{k+1} \\ y_{k+1} \\ \theta_{k+1} \end{bmatrix} = \begin{bmatrix} x_k + R\bigl(\sin(\theta_k + \Delta\theta) - \sin\theta_k\bigr) \\ y_k - R\bigl(\cos(\theta_k + \Delta\theta) - \cos\theta_k\bigr) \\ \theta_k + \Delta\theta \end{bmatrix}, \quad \Delta\theta = \dot{\theta}\,\Delta t$$

Degenerate cases: straight line ($\omega_R = \omega_L$) → $\Delta x = v\Delta t\cos\theta,\ \Delta y = v\Delta t\sin\theta$; pure spin ($\omega_R = -\omega_L$) → only $\theta$ changes

Each block can be dropped directly into any LaTeX document wrapped in `\[ ... \]` for display math or `$ ... $` for inline math.

SYMBOL REFERENCE

| | | | |
|---|---|---|---|
| $x, y$ | World-frame position [m] | $r$ | Wheel radius [m] |
| $\theta$ | Heading angle (yaw) [rad] | $b$ | Wheelbase [m] |
| $\omega_R, \omega_L$ | Wheel ang. velocities [rad/s] | $R$ | ICC radius [m] |
| $v$ | Linear velocity [m/s] | $\Delta t$ | Timestep [s] |

**State vector** — the robot's pose is a 3D vector (x,y,θ)(x, y, \theta) (x,y,θ): planar position in the world frame and the heading angle. This is your "state" in the robotics/control sense.

**Control inputs** — the raw actuator commands are the two wheel angular velocities ωR\omega_R ωR and ωL\omega_L ωL. Everything else is derived from these.

**Velocity kinematics** — the wheel velocities are first converted to body-frame quantities: linear speed vv v (average of both wheels) and rotational rate θ˙\dot\theta θ˙ (difference of both wheels). This is the core differential-drive insight.

**Continuous-time model** — the three ODEs form the nonholonomic unicycle model. The cosθ\cos\theta cosθ and sinθ\sin\theta sinθ coupling is what makes it nonlinear, and what makes dead-reckoning accumulate errors over time.

**Discrete update (Euler)** — this is what you implement in a real control loop. It's an approximation valid when Δt\Delta t Δt is small.

**Exact arc integration (ICC)** — instead of linearising, you compute the true circular arc swept during Δt\Delta t Δt. This gives zero numerical integration error for constant inputs, which is why odometry libraries prefer it. The robot pivots around an Instantaneous Centre of Curvature at distance RR R from the midpoint of the axle.

Let me know if you'd like to extend this to 3D pose (roll/pitch/yaw), add wheel slip/noise models for EKF-SLAM, or adapt it to a different drive type (Mecanum, Ackermann, etc.).

**ROS 2–compatible kinematic formulation:**
This is an important modelling decision to get right before writing a single equation. A differential drive robot is a ground vehicle — its wheels constrain it to a surface. Extending naively to 6-DOF 3D is only physically meaningful in two scenarios: the ground itself is non-flat (a ramp, rough terrain), or you're modelling a robot that can genuinely move in 3D (an underwater vehicle, a drone). The treatment below covers the most practically useful case: a differential drive robot on uneven terrain, where (z,roll,pitch)(z, \text{roll}, \text{pitch}) (z,roll,pitch) are driven by the surface, not by independent actuators.

### Extended state vector and control inputs:
The 3D pose is a 6-DOF vector. Control inputs remain the two wheel angular velocities — the robot has no independent actuator for z , roll, or pitch.

$$\mathbf{q}(t) = \begin{bmatrix} x(t)\\ y(t)\\ z(t)\\ \phi(t)\\ \theta(t)\\ \psi(t) \end{bmatrix}, \qquad$$

$$\mathbf{u}(t) = \begin{bmatrix} \omega_L(t)\\ \omega_R(t) \end{bmatrix}$$

$$\mathbf{q}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \\ \phi(t) \\ \theta(t) \\ \psi(t) \end{bmatrix}, \qquad \mathbf{u}(t) = \begin{bmatrix} \omega_L(t) \\ \omega_R(t) \end{bmatrix} \tag{1}$$

| | | |
|---|---|---|
| $x, y, z$ | state | Position in world frame [m] |
| $\phi$ | state | Roll — rotation about world $x$-axis [rad] |
| $\theta$ | state | Pitch — rotation about world $y$-axis [rad] |
| $\psi$ | state | Yaw — rotation about world $z$-axis (heading) [rad] |
| $\omega_L, \omega_R$ | input | Wheel angular velocities [rad s⁻¹] |

**Step 1 — wheel velocities to body-frame speeds**

Identical to the 2D case. The forward speed $v$ and yaw rate $\psi$ come entirely from the wheel mixing equations.

v(t) = \frac{r}{2}\bigl(\omega_R + \omega_L\bigr),

\qquad

\dot{\psi}(t) = \frac{r}{L}\bigl(\omega_R - \omega_L\bigr)

$$v(t) = \frac{r}{2}\left(\omega_R + \omega_L\right), \qquad \dot{\psi}(t) = \frac{r}{L}\left(\omega_R - \omega_L\right) \tag{2}$$

| | | |
|---|---|---|
| $r$ | param | Wheel radius [m] |
| $L$ | param | Track width (wheel-to-wheel distance) [m] |
| $v$ | derived | Forward speed in the body frame [m s⁻¹] |
| $\dot{\psi}$ | derived | Yaw rate [rad s⁻¹] |

### Step 2 — rotation matrix (ZYX / extrinsic)

To project body-frame velocity into world-frame velocity, we need the full 3D rotation matrix. Using the ZYX (yaw-pitch-roll) convention, composing three elementary rotations $R_z(\psi)\, R_y(\theta)\, R_x(\phi)$:

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix} \tag{3}$$

where $c_\alpha \equiv \cos\alpha$, $s_\alpha \equiv \sin\alpha$. The first column of $R$ is the body $x$-axis (forward direction) expressed in world coordinates — exactly what we need to project $v$.

R =
\begin{bmatrix}
c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\
s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\
- s_\theta & c_\theta s_\phi & c_\theta c_\phi
\end{bmatrix}

### Step 3 — world-frame translational velocity

The robot moves forward at speed $v$ along its body $x$-axis. Multiplying by the first column of $R$ gives all three world-frame velocity components simultaneously.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = v(t) \cdot R(\phi, \theta, \psi)\, \mathbf{e}_1 = v(t) \begin{bmatrix} c_\psi c_\theta \\ s_\psi c_\theta \\ -s_\theta \end{bmatrix} \tag{4}$$

**Interpreting $\dot{z}$:** The vertical velocity is $\dot{z} = -v \sin\theta$. When the robot drives uphill ($\theta > 0$), $\dot{z} > 0$. On flat ground $\theta = 0$, so $\dot{z} = 0$ and we recover the 2D equations. The robot cannot change $z$ except by driving along a surface whose pitch changes.

\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix}
=
v(t) \begin{bmatrix} c_\psi c_\theta \\ s_\psi c_\theta \\ - s_\theta \end{bmatrix}

### Step 4 — Euler angle rates (angular velocity kinematics)

The body angular velocity $\boldsymbol{\omega}_b = (p,\ q,\ r)^\top$ and the Euler angle rates $(\dot{\phi}, \dot{\theta}, \dot{\psi})^\top$ are related by the Euler kinematic equations. For a diff-drive robot, only yaw rate $\dot{\psi}$ is actuated; roll and pitch rates come from the terrain.

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & s_\phi t_\theta & c_\phi t_\theta \\ 0 & c_\phi & -s_\phi \\ 0 & s_\phi / c_\theta & c_\phi / c_\theta \end{bmatrix}}_{T(\phi,\theta)^{-1}} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \tag{5}$$

| | |
|---|---|
| $t_\alpha$ | $\tan\alpha$ |
| $p, q, r$ | derived Roll, pitch, yaw rates in the body frame |
| Singularity | Eq. (5) is singular at $\theta = \pm 90^\circ$. For steep terrain, replace with quaternion representation. |

For a ground robot with surface contact, the body angular velocity components are constrained as:

$$p = \dot{\phi}_{\text{surface}}, \quad q = \dot{\theta}_{\text{surface}}, \quad r = \dot{\psi} \tag{6}$$

where $\dot{\phi}_{\text{surface}}$ and $\dot{\theta}_{\text{surface}}$ are the time-derivatives of the terrain surface normal projected onto the robot body, and $\dot{\psi}$ comes from eq. (2).

```
\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}
=
\begin{bmatrix}
1 & s_\phi t_\theta & c_\phi t_\theta \\
0 & c_\phi & -s_\phi \\
0 & s_\phi / c_\theta & c_\phi / c_\theta
\end{bmatrix}

\begin{bmatrix} p \\ q \\ r \end{bmatrix}
```

### Step 5 — surface constraint (terrain model)

On a surface described by height map $z = h(x, y)$, roll and pitch are determined by the local terrain gradient, not by the actuators. Let $\nabla h = (\partial h/\partial x,\ \partial h/\partial y)$:

$$\theta(t) = -\arctan\!\left( \frac{\partial h}{\partial x}\cos\psi + \frac{\partial h}{\partial y}\sin\psi \right) \tag{7a}$$

\theta(t) = - \arctan\!\left( \frac{\partial h}{\partial x}\cos\psi + \frac{\partial h}{\partialy}\sin\psi \right)|

$$\phi(t) = \arctan\!\left( \frac{ -\frac{\partial h}{\partial x}\sin\psi + \frac{\partial h}{\partial y}\cos\psi}{ \sqrt{ 1 + \left( \frac{\partial h}{\partial x}\cos\psi + \frac{\partial h}{\partialy}\sin\psi \right)^2 } } \right) \tag{7b}$$

\phi(t) = \arctan\!\left( \frac{ -\frac{\partial h}{\partial x}\sin\psi + \frac{\partial h}{\partial y}\cos\psi}{ \sqrt{ 1 + \left( \frac{\partial h}{\partial x}\cos\psi + \frac{\partial h}{\partialy}\sin\psi \right)^2 } } \right)

$$z(t) = h\big(x(t),\ y(t)\big) \tag{7c}$$

Equations (7a–c) mean $z, \phi, \theta$ are *algebraically constrained* by the terrain at each step — they are not freely integrated. Only $x, y, \psi$ are independently integrated from the wheel inputs.

z(t) = h\bigl(x(t), y(t)\bigr)

### Complete discrete-time update (Euler integrat

Combining all steps, the full 6-DOF state update over timestep $\Delta t$ is:

$$\boxed{\begin{aligned} x_{k+1} &= x_k + v_k\, c_{\psi_k} c_{\theta_k}\, \Delta t \\ y_{k+1} &= y_k + v_k\, s_{\psi_k} c_{\theta_k}\, \Delta t \\ z_{k+1} &= h(x_{k+1},\, y_{k+1}) \\ \phi_{k+1} &= \phi_{\text{surface}}(x_{k+1}, y_{k+1}, \psi_{k+1}) \\ \theta_{k+1} &= \theta_{\text{surface}}(x_{k+1}, y_{k+1}, \psi_{k+1}) \\ \psi_{k+1} &= \psi_k + \dot{\psi}_k\, \Delta t \end{aligned}} \tag{8}$$

where $v_k$ and $\dot{\psi}_k$ come from eq. (2), and the surface functions $h$, $\phi_{\text{surface}}$, $\theta_{\text{surface}}$ come from eqs. (7a–c). This is the minimum viable 3D extension: only $x$, $y$, $\psi$ are numerically integrated; the remaining three states are read off the terrain at each step.

```
\begin{aligned}
x_{k+1} &= x_k + v_k\, c_{\psi_k} c_{\theta_k}\,\Delta t \\
y_{k+1} &= y_k + v_k\, s_{\psi_k} c_{\theta_k}\,\Delta t \\
z_{k+1} &= h(x_{k+1}, y_{k+1}) \\
\phi_{k+1} &= \phi_{\text{surface}}(x_{k+1}, y_{k+1}, \psi_{k+1}) \\
\theta_{k+1} &= \theta_{\text{surface}}(x_{k+1}, y_{k+1}, \psi_{k+1}) \\
\psi_{k+1} &= \psi_k + \dot{\psi}_k \Delta t
\end{aligned}
```

### Unconstrained 3D extension (6-DOF free body — drone / AUV)

If the robot is not surface-constrained (aerial, underwater), all six states must be integrated. The body-frame velocity vector becomes $\mathbf{v}_b = (v_x^b, v_y^b, v_z^b)^\top$ with independent actuation on all axes:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} = R(\phi, \theta, \psi) \begin{bmatrix} v_x^b \\ v_y^b \\ v_z^b \end{bmatrix}, \qquad \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = T(\phi, \theta)^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix} \tag{9}$$

For this model all six angular velocities $(p, q, r)$ and all three linear velocities $(v_x^b, v_y^b, v_z^b)$ are independent inputs — none are coupled by a wheel constraint. This is the standard rigid-body kinematic model used in UAV and AUV navigation.

```
\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \end{bmatrix} =
R(\phi,\theta,\psi) \begin{bmatrix} v_x^b \\ v_y^b \\ v_z^b \end{bmatrix}

\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} =
```

T(\phi,\theta)^{-1} \begin{bmatrix} p \\ q \\ r \end{bmatrix}

**Key differences: 2D → 3D at a glance**

**What stays the same**

Wheel mixing equations for $v$ and $\dot{\psi}$ are identical. The $x$–$y$ update gains a $\cos\theta$ factor to account for slope, but the structure is unchanged.

**What is new**

Full $3 \times 3$ rotation matrix $R$. Euler kinematic matrix $T^{-1}$. Terrain height map coupling $z, \phi, \theta$ to position. Singularity at $\theta = \pm 90^\circ$.

**3 integrated states**

$x, y, \psi$ — driven directly by $\omega_L, \omega_R$ via eqs. (2) and (8).

**3 constrained states**

$z, \phi, \theta$ — set algebraically by the terrain at each step, not integrated independently.

Extending to 3D does not give you six free degrees of freedom — a wheeled ground robot only has two actuated ones (ωL,ωR\omega_L, \omega_R ωL,ωR), the same as before. The 3D extension changes the *projection* of those inputs into world coordinates (via the full rotation matrix RR R), and adds a *constraint subsystem* that reads zz z, ϕ\phi ϕ, θ\theta θ from the terrain rather than integrating them freely.

Three things are worth keeping in mind for implementation:

**The cosθ\cos\theta cosθ factor in the xx x–yy y update** is subtle but important. On a 30° slope, cos(30°)≈0.87\cos(30°) \approx 0.87 cos(30°)≈0.87, meaning the robot covers 13% less horizontal ground per metre of wheel travel than the odometry naively suggests. Ignoring this is a common source of dead-reckoning drift in hilly environments.

**The Euler kinematic matrix T−1T^{-1} T−1 (eq. 5) is singular at θ=±90°\theta = \pm 90° θ=±90°** — i.e. a perfectly vertical surface. For most outdoor ground robots this never occurs, but if you're operating on steep embankments and need robustness, replace the Euler angle representation with a unit quaternion q=(q0,q1,q2,q3)⊤\mathbf{q} = (q_0, q_1, q_2, q_3)^\top q=(q0,q1,q2,q3)⊤ and use the quaternion kinematics q˙=12Ξ(q) ωb\dot{\mathbf{q}} = \frac{1}{2}\Xi(\mathbf{q})\,\boldsymbol\omega_b q˙=21Ξ(q)ωb instead — it has no singularities anywhere.

**The terrain model h(x,y)h(x,y) h(x,y) is the hardest part in practice**. In simulation you have it analytically. On a real robot you either build it from a prior map, estimate it from an IMU (which measures ϕ\phi ϕ, θ\theta θ directly, making the surface-constraint equations redundant), or fuse wheel odometry with IMU in an EKF — which is the standard approach for 3D ground robot localisation.